

SPIN - Seminar Robert's

–

Formal Methods for AI-Based Systems Engineering (FMAISE)

Robert Li & William Tossici (1983696, 1844504)

Academic Year 2025/2026

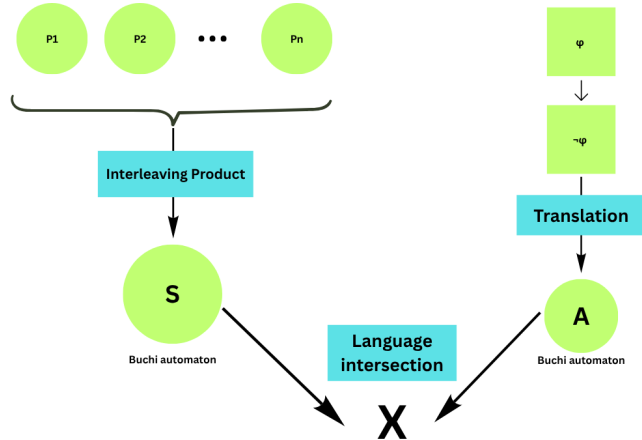


SAPIENZA
UNIVERSITÀ DI ROMA



The Automata-Theoretic Approach

O

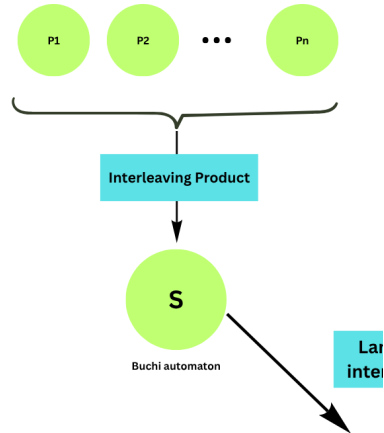




The Automata-Theoretic Approach

O

- Each process template translates into FSA
- Computing async interleaving product of FSA
- Resulting global system is again an automaton





Basic Depth-First Search (State Space Exploration)

○

Objective: Visit every global state reachable from the initial state s_0 .

Key Characteristics:

- **Duplicate Prevention:** The Statespace (V) ensures no state is visited twice.
- **Memory Optimization:** The algorithm stores *only states* (nodes), deliberately ignoring transitions (edges).
- **On-The-Fly Generation:** In tools like SPIN, the transition relation ($A.T$) is not precomputed. Successors are calculated dynamically from individual asynchronous components.

Basic DFS Algorithm

```
Stack D = {}
Statespace V = {}

Start() {
  Add_Statespace(V, A.s0)
  Push_Stack(D, A.s0)
  Search()
}

Search() {
  s = Top_Stack(D)
  for each (s, l, s') ∈ A.T
    if In_Statespace(V, s') == false {
      Add_Statespace(V, s')
      Push_Stack(D, s')
      Search()
    }
  Pop_Stack(D)
}
```



Checking Safety Properties in DFS

○

Property 8.1: Finite Termination

- The search always terminates because the global state space ($A.S$) is finite, and V prevents infinite loops.



Checking Safety Properties in DFS

○

Property 8.1: Finite Termination

- The search always terminates because the global state space ($A.S$) is finite, and V prevents infinite loops.

The Safety Evaluation

- **Deadlocks:** Does state s have successors?
- **Assertions:** Are local invariants (e.g., $a > b$) valid at state s ?



Checking Safety Properties in DFS

○

Property 8.1: Finite Termination

- The search always terminates because the global state space ($A.S$) is finite, and V prevents infinite loops.

The Safety Evaluation

- **Deadlocks:** Does state s have successors?
- **Assertions:** Are local invariants (e.g., $a > b$) valid at state s ?

Generating Counter-Examples

- An error message is useless without the *path* that caused it.
- The Stack (D) naturally holds the exact execution trace from s_0 to the error.

Caveat: DFS traces are valid, but not necessarily the *shortest* possible paths.

Extended DFS (Figure 8.2)

```
Search() {  
  s = Top_Stack(D)  
  if !Safety(s) {  
    Print_Stack(D)  
  }  
  for each (s, l, s') ∈ A.T  
    if In_Statespace(V, s') == false {  
      Add_Statespace(V, s')  
      Push_Stack(D, s')  
      Search()  
    }  
  Pop_Stack(D)  
}
```



Computational Requirements & State Explosion

○

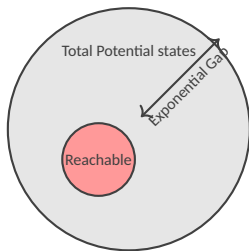
Complexity Analysis

- **Linear in Reachable States:** The DFS is $O(N + M)$, where N is the number of reachable states and M is the number of transitions.
- **The Cartesian Product:** For n asynchronous processes, the total state space is:

$$|A.S| = \prod_{i=1}^n |A_i.S|$$

- **Exponential Growth:** Adding even one small process can multiply the state space size by its local states.

The "Fraction" Reality



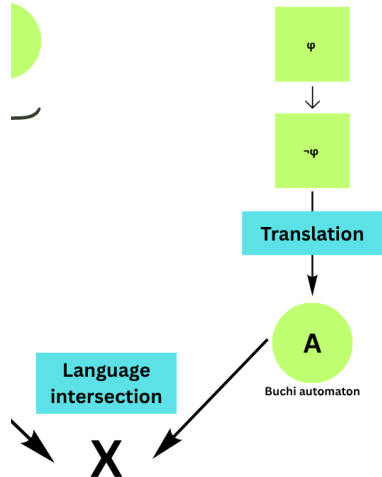
"A small fraction of an astronomical number is still very large."



The Automata-Theoretic Approach

○

- Take correctness claim specified as LTL formula
- Convert formula into Buchi automaton
- Compute synchronous product of this claim and the global state space automaton
- Result is another Buchi automaton, check language intersection





The Automata-Theoretic Approach

○

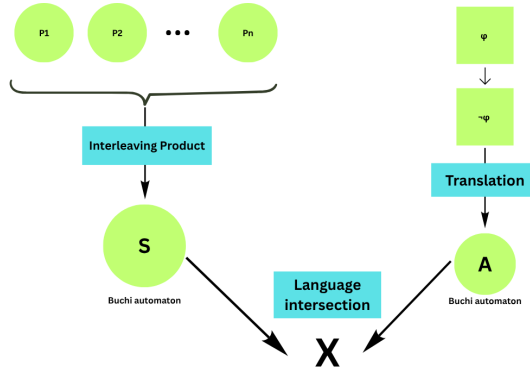




Table of Contents

1 Search for a cycle: classical algorithm

- ▶ Search for a cycle: classical algorithm
- ▶ Search for a cycle: NDFS + bit-state hashing
- ▶ NDFS: reduction techniques
- ▶ NDFS: incompatibilities
- ▶ NDFS: corrections



Tarjan's DFS algorithm

1 Search for a cycle: classical algorithm

- Classical algorithm for finding cycles
- Constructs SCCs of a graph in linear time
- Adds two integer numbers to each state: *dfs*-number and *lowlink*-number
- Result is another Buchi automaton, check language intersection



Tarjan's DFS algorithm

1 Search for a cycle: classical algorithm

- Classical algorithm for finding cycles
- Constructs SCCs of a graph in linear time
- Adds two integer numbers to each state: *dfs-number* and *lowlink-number*
- Result is another Buchi automaton, check language intersection



Tarjan's algorithm: memory issues

1 Search for a cycle: classical algorithm

Why this is a problem for SPIN:

- **High Memory Overhead:** Storing two 32-bit integers per state. State space that SPIN can generate may contain billions of reachable states!
- **Hashing Incompatibility:** Bit-state hashing (next seminars topic) is designed to store only *bits*. Tarjan requires full state vectors to store lowlink values.

Tarjan's DFS

- Find cycles via SCCs
- Linear time complexity
- Requires *dfs* & *lowlink* integers
- Language intersection check



Table of Contents

2 Search for a cycle: NDFS + bit-state hashing

- ▶ Search for a cycle: classical algorithm
- ▶ Search for a cycle: NDFS + bit-state hashing
- ▶ NDFS: reduction techniques
- ▶ NDFS: incompatibilities
- ▶ NDFS: corrections



Nested Depth First Search

2 Search for a cycle: NDFS + bit-state hashing

NDFS + bit-state hashing:

- Solve emptiness problem w.o explicitly constructing the SCC

From: Courcoubetis et al., *Memory Efficient Algorithms for the Verification of Temporal Properties*, FMSD 1992.



Nested Depth First Search

2 Search for a cycle: NDFS + bit-state hashing

NDFS + bit-state hashing:

- Solve emptiness problem w.o explicitly constructing the SCC
- The nested depth-first search algorithm does not preserve the capability to detect all possible acceptance cycles that may appear in the reachability graph.

From: Courcoubetis et al., *Memory Efficient Algorithms for the Verification of Temporal Properties*, FMSD 1992.



Nested Depth First Search

2 Search for a cycle: NDFS + bit-state hashing

NDFS + bit-state hashing:

- Solve emptiness problem w.o explicitly constructing the SCC
- The nested depth-first search algorithm does not preserve the capability to detect all possible acceptance cycles that may appear in the reachability graph.
- It can, however, be proven to detect at least one such cycle if one or more cycles exists

From: Courcoubetis et al., *Memory Efficient Algorithms for the Verification of Temporal Properties*, FMSD 1992.



Nested Depth First Search

2 Search for a cycle: NDFS + bit-state hashing

NDFS + bit-state hashing:

- Solve emptiness problem w.o explicitly constructing the SCC
- The nested depth-first search algorithm does not preserve the capability to detect all possible acceptance cycles that may appear in the reachability graph.
- It can, however, be proven to detect at least one such cycle if one or more cycles exists
- Fast hash-table lookup. The table can be stored in the main memory.
- We allow the algorithm to err with small probability, but reduce Space Complexity: $O(n)$ vs $O(n \log n)$ where n is states.

From: Courcoubetis et al., *Memory Efficient Algorithms for the Verification of Temporal Properties*, FMSD 1992.



Sequential 2-Pass NDFS Algorithm

2 Search for a cycle: NDFS + bit-state hashing

Core Data Structures:

- S : Stack holding the current DFS path (root to current node).
- Q : FIFO queue holding reachable accepting states ($v \in F$) in *postorder*.
- M : Bit-array indexed by hash values to track "marked" (visited) states.

Pass 1: Postorder Collection

```
Initialize:  $S := [s_0], M := 0, Q := \emptyset$   
while  $S \neq \emptyset$  do:  
     $v := \text{top}(S)$   
    if all  $w \in \text{succ}(v)$  have  $M[h(w)] = 1$ :  
        pop  $v$  from  $S$   
        if  $v \in F$  then insert  $v$  into  $Q$   
    else:  
        let  $w$  be the first unmarked successor  
         $M[h(w)] := 1$   
        push  $w$  into  $S$ 
```

13/24

Pass 2: Cycle Detection

```
Initialize:  $S := \emptyset, M := 0$   
while  $Q \neq \emptyset$  do:  
     $f := \text{head}(Q)$ , pop  $Q$ , push  $f \rightarrow S$   
    while  $S \neq \emptyset$  do:  
         $v := \text{top}(S)$   
        if  $f \in \text{succ}(v)$  then return "YES"  
        if all  $w \in \text{succ}(v)$  are marked:  
            pop  $v$  from  $S$   
        else:  
            mark and push first unmarked  $w$  to  $S$ 
```



Correctness: Lemma 1 and Theorem 1

2 Search for a cycle: NDFS + bit-state hashing

Lemma 1: The Postorder Property

- Let $f_1, \dots, f_k$ be accepting states in **postorder** (finished order).
- If there is a path $f_i \rightarrow f_j$ where $i < j$, then f_i is in a cycle.

Proof Sketch: If f_j finishes **after** f_i , yet f_i can reach f_j , then f_j must have been an **ancestor** of f_i when f_i was first visited. If you can reach your ancestor, you are in a cycle.

Theorem 1: Soundness and Completeness

- **Soundness:** If DFS 2 returns “YES”, a cycle exists.
 - *Proof:* DFS 2 only returns “YES” if it finds a physical back-edge to the current root; this is a literal cycle.
- **Completeness:** If a cycle exists, DFS 2 will find it.
 - *Proof:* Let f_j be the **first** reachable accepting state in a cycle (smallest index). Because it is the first, no other search has “stolen” its nodes yet.
 - Thus, its path back to itself is unmarked and *guaranteed* to be explored, triggering the “YES”.



Interleaved NDFS (Two Stacks)

2 Search for a cycle: NDFS + bit-state hashing

- If the automaton is found to be nonempty, an accepted word can be extracted from the stacks $S1$ and $S2$.
- Hash collision: The algorithm might erroneously conclude (due to collisions) that the Buchi automaton does not accept any word, but will never mistakenly conclude that the automaton accepts some word.
- "In terms of the underlying verification problem, this means that our algorithm might miss some errors, but will never falsely claim that the protocol is incorrect. Thus, the algorithm should be viewed more as a systematic debugging tool rather than as a verification tool."

From: Courcoubetis et al., *Memory Efficient Algorithms for the Verification of Temporal Properties*, FMSD 1992.



Table of Contents

3 NDFS: reduction techniques

- ▶ Search for a cycle: classical algorithm
- ▶ Search for a cycle: NDFS + bit-state hashing
- ▶ **NDFS: reduction techniques**
- ▶ NDFS: incompatibilities
- ▶ NDFS: corrections



Initialization

3 NDFS: reduction techniques

- The statespace that is constructed with the depth-first search procedure can be reduced substantially if we limit the number of successor states that is explored from each reachable state.



Initialization

3 NDFS: reduction techniques

- The statespace that is constructed with the depth-first search procedure can be reduced substantially if we limit the number of successor states that is explored from each reachable state.
- We can implement POR technique.



Table of Contents

4 NDFS: incompatibilities

- ▶ Search for a cycle: classical algorithm
- ▶ Search for a cycle: NDFS + bit-state hashing
- ▶ NDFS: reduction techniques
- ▶ **NDFS: incompatibilities**
- ▶ NDFS: corrections



POR C_3 Conflict in Nested DFS (Algorithm B)

4 NDFS: incompatibilities

The Visibility Gap:

- **The C_3 Vulnerability:** Condition C_3 forbids a reduced **ample set** if it closes a cycle on the *current* search stack.
- **Stack Asymmetry:** Phase 2 (Red Search) evaluates C_3 strictly against its own stack (S_2), remaining unaware of the paused Blue Stack (S_1).
- **Fatal Ample Set:** At state v , a transition to $w \in S_1$ is ignored since $w \notin S_2$.
- **Result: A False Negative** (missed cycle) as the cycle-closing transition is omitted.

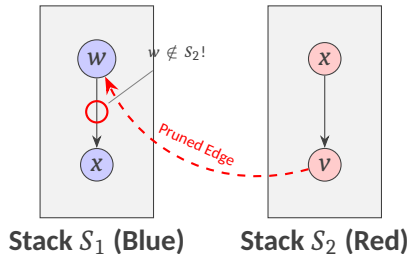




Table of Contents

5 NDFS: corrections

- ▶ Search for a cycle: classical algorithm
- ▶ Search for a cycle: NDFS + bit-state hashing
- ▶ NDFS: reduction techniques
- ▶ NDFS: incompatibilities
- ▶ **NDFS: corrections**



The Correction: Stack Intersection Strategy

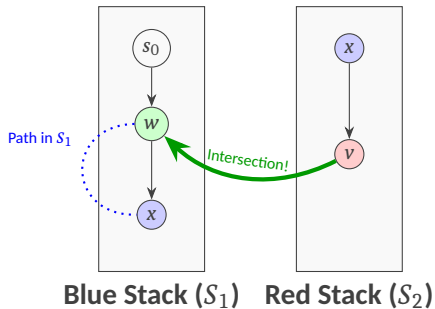
5 NDFS: corrections

The Concept:

- Instead of hunting for the *Seed* state, DFS 2 hunts for the **Blue Stack** (S_1).
- **Termination Rule:** If DFS 2 reaches any state $w \in S_1$, halt and return “YES”.

Why this fixes POR:

- DFS 2 never explores *beyond* the Blue Stack.
- The “Blind Spot” is removed; the search terminates exactly where the C_3 condition would have failed.
- Eliminates the need to store the seed state.





Correction 5.2: Preserving Search Consistency

5 NDFS: corrections

The Problem: The Dynamic Graph

- POR decisions are **stack-dependent**.
- S_1 and S_2 often reach the same state via different paths.
- Without correction, Phase 2 might explore a “different” graph than Phase 1.

The Solution: Memoization of POR Choices

- The Blue Search (Phase 1) acts as the “Decision Maker.”
- It stores a **Selection Number** or **Bit** in the state descriptor.
- **Phase 2 Requirement:** The Red Search must **replay** the exact same transition choices as the Blue Search.

The “Leader-Follower” Principle

By forcing the Red Search to ignore its own stack context and follow the Blue Search’s recorded decisions, we ensure both searches operate on a **consistent, deterministic subgraph**.



SPIN Implementation: NDFS & POR Integration

5 NDFS: corrections

1. The 3-Bit State Metadata Header

To minimize memory footprint, SPIN packs the algorithm's state directly into the global hash table. Each state descriptor utilizes exactly **3 bits**:

- **Bits 0 & 1:** Track visitation for the Blue and Red searches.
- **Bit 2 (Selection Bit):** Stores Phase 1's POR decision. Phase 2 reads this bit to ensure it traverses the *exact same graph*, bypassing its own stack-dependent C_3 check.



Complexity: Safety vs. Liveness Properties

5 NDFS: corrections

1. Time Complexity (The $2\times$ Factor)

- **Safety:** Linear in the number of reachable states ($O(V + E)$).
- **Liveness:** Requires cycle detection (NDFS), which visits states in two phases (Blue and Red searches).
- **Result:** Worst-case runtime is exactly **twice** that of a safety check.

2. Memory Complexity (The 2-Bit Trick)

- Rather than duplicating the graph, SPIN adds **two bits** per state:
 - **Bit 1:** State visited in Phase 1 (Blue search).
 - **Bit 2:** State visited in Phase 2 (Red search).

3. The Property Automaton (N)

- Verifying an LTL property adds a multiplier N (states in the *never claim*).
- **Total Statespace:** $S \times N$.